

UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

Departamento de Computación, Electrónica y Mecatrónica

Reporte de práctica

Práctica 1: Unidad Aritmética Lógica (ALU) Simple de 4 bits

Materia	P26-LIS2022-1 Arquitecturas Computacionales
Profesor	Berenice Rodriguez Pedroza
Sección	2
Periodo	Primavera 2026
Fecha de práctica	4 de marzo de 2026
Fecha de entrega	5 de marzo de 2026

Integrantes

Robbie Nicolás Curioso de Salazar	186028	robbie.curiosodr@udlap.mx
Hector López Sanchez	185979	hector.lopezsz@udlap.mx

1. Introducción

Una Unidad Aritmética Lógica (ALU) es el bloque fundamental de ejecución dentro de un procesador, encargado de realizar cálculos aritméticos y evaluaciones lógicas. Esta práctica se desarrolla como una introducción al diseño de unidades funcionales en hardware descriptivo utilizando herramientas de simulación y síntesis FPGA.

De acuerdo con la práctica, la ALU selecciona una operación mediante una señal de control y produce un resultado de 4 bits. La validación se realizó en montaje físico con protoboard, switches y LEDs conectados a los puertos PMOD de la tarjeta PYNQ-Z1.

2. Objetivos

- Diseñar e implementar una ALU simple de 4 bits con operaciones aritméticas y lógicas básicas.
- Describir el diseño en VHDL utilizando un proceso combinacional.
- Sintetizar e implementar el diseño en Vivado para la FPGA Zynq-7000.
- Validar el funcionamiento en hardware real usando protoboard, switches y LEDs.

3. Materiales y herramientas

- Software: Vivado 2020.1 o superior.

- Hardware: Tarjeta PYNQ-Z1 (xc7z020c1g400-1).
- Protoboard, switches (entradas) y LEDs (salidas).
- Cable USB para programación y alimentación.

4. Marco teórico

4.1. Unidad Aritmética Lógica

La ALU constituye el bloque de ejecución dentro de un procesador, ya que permite realizar cálculos aritméticos y evaluaciones lógicas. En arquitecturas RISC como RISC-V, la ALU es puramente combinacional: no tiene reloj ni memoria interna, y la salida cambia inmediatamente al cambiar las entradas.

4.2. Especificación de la práctica

La ALU recibe:

- $A[3 : 0]$: Operando A de 4 bits.
- $B[3 : 0]$: Operando B de 4 bits.
- `alu_ctrl`: Señal de selección de operación.

Y produce:

- $Result[3 : 0]$: Resultado de la operación.

4.3. Operaciones implementadas

En esta implementación se seleccionan operaciones mediante `alu_ctrl` (4 bits). Para la

validación en hardware se probaron las operaciones de suma, resta y operaciones lógicas básicas.

Cuadro 1: Operaciones utilizadas en las pruebas de la práctica.

alu_ctrl	Operación	Descripción
0000	ADD	Suma sin signo
0001	SUB	Resta ($A - B$)
0010	AND	AND bit a bit
0011	OR	OR bit a bit
0100	XOR	XOR bit a bit
0101	NOT	NOT sobre A

4.4. Descripción de las operaciones

Suma sin signo (ADD):

$$R = A + B$$

Resta (SUB): Se implementa como $A - B$ (equivalente a $A + (\sim B + 1)$ en complemento a dos).

Operaciones lógicas: Se aplican bit a bit entre los operandos:

$$\text{AND: } R_i = A_i \wedge B_i \quad \text{OR: } R_i = A_i \vee B_i$$

$$\text{XOR: } R_i = A_i \oplus B_i \quad \text{NOT: } R_i = \overline{A_i}$$

4.5. PMOD y mapeo de pines

Los conectores PMOD de la PYNQ-Z1 proporcionan pines de E/S a 3.300 V (LVCMOS33). Se usó PMODA para las entradas (A y B) y PMODB para el control y la salida.

5. Diseño y procedimiento

5.1. Implementación en VHDL

El módulo `alu1_0` se implementó con un proceso combinacional sensible a las entradas. Dentro del proceso, una sentencia `case` selecciona la operación según la señal de control.

Se utilizó `IEEE.NUMERIC_STD` con el tipo `unsigned` para las operaciones aritméticas, y operadores lógicos nativos de `std_logic_vector` para AND, OR, XOR y NOT.

5.2. Asignación de pines (XDC)

Se creó el archivo de restricciones `alu4bits_pynqz1_pmod.xdc` con el siguiente mapeo:

Cuadro 2: Asignación de pines PMOD.

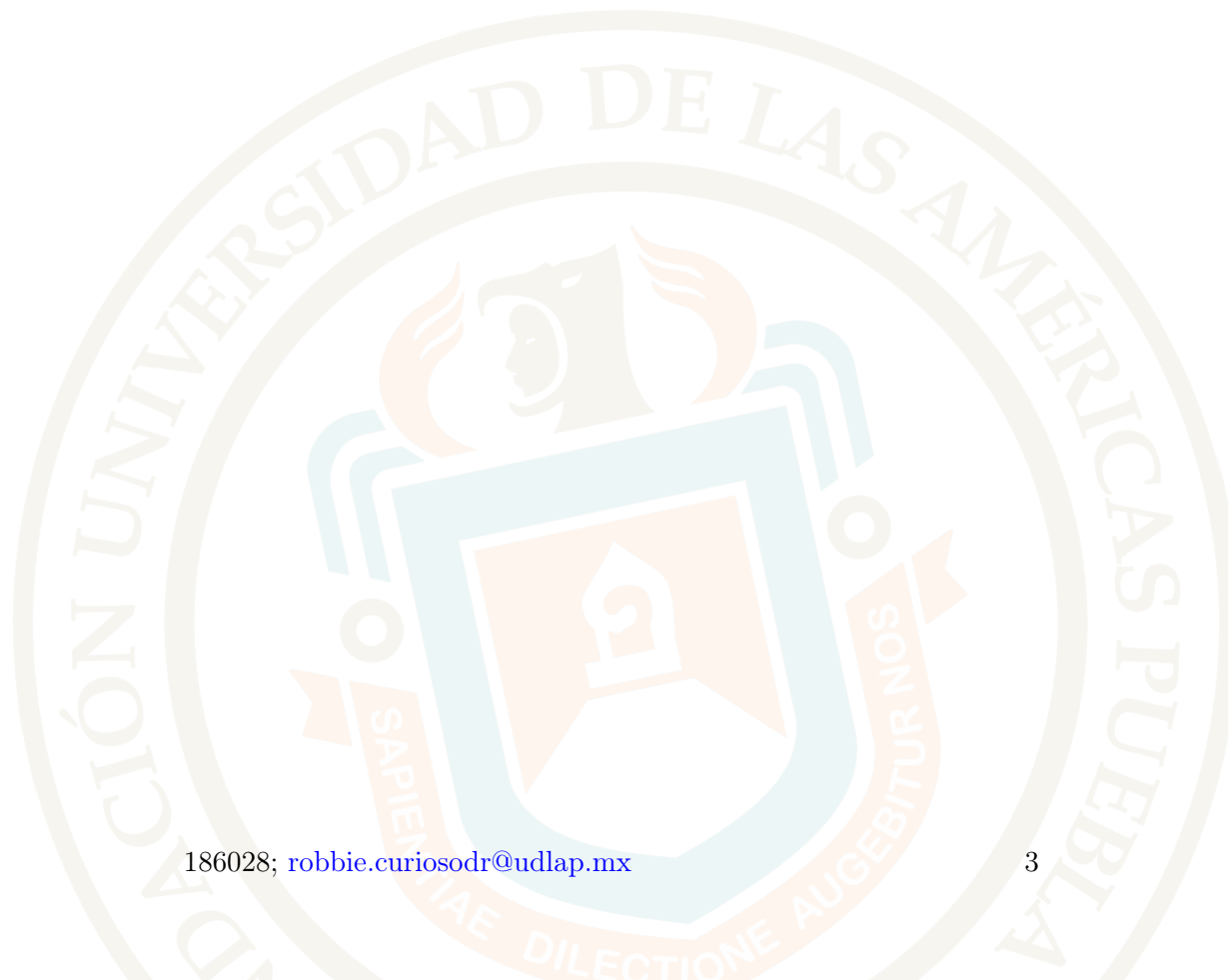
Puerto	Conector	Pines
<code>op_a[3:0]</code>	PMODA[0..3]	Y18, Y19, Y16, Y17
<code>op_b[3:0]</code>	PMODA[4..7]	U18, U19, W18, W19
<code>alu_ctrl[2:0]</code>	PMODB[0..2]	W14, Y14, T11
<code>result[3:0]</code>	PMODB[4..7]	V16, W16, V12, W13

Se configuraron todos los pines con LVCMOS33 y se habilitaron pull-ups en las entradas superiores para evitar lecturas flotantes.

5.3. Flujo de implementación

1. Diseñar la ALU utilizando un proceso combinacional.
2. Utilizar una sentencia `case` para seleccionar la operación según `alu_ctrl`.
3. Crear proyecto en Vivado con el módulo ALU como top module.

4. Agregar archivo de restricciones XDC.
5. Ejecutar síntesis, implementación y generar bitstream.
6. Programar la PYNQ-Z1 y conectar protoboard.
7. Verificar el funcionamiento mediante pruebas en hardware.



6. Resultados y discusión

6.1. Código VHDL de la ALU

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity alu1_0 is
6      Port (
7          op_a      : in  STD_LOGIC_VECTOR(3 downto 0);
8          op_b      : in  STD_LOGIC_VECTOR(3 downto 0);
9          alu_ctrl  : in  STD_LOGIC_VECTOR(3 downto 0);
10         result    : out STD_LOGIC_VECTOR(3 downto 0)
11     );
12 end alu1_0;
13
14 architecture Behavioral of alu1_0 is
15 begin
16     process(op_a, op_b, alu_ctrl)
17     begin
18         case alu_ctrl is
19             when "0000" => -- ADD (suma sin signo)
20                 result <= std_logic_vector(unsigned(op_a) + unsigned(op_b));
21             when "0001" => -- SUB (resta)
22                 result <= std_logic_vector(unsigned(op_a) - unsigned(op_b));
23             when "0010" => -- AND
24                 result <= op_a and op_b;
25             when "0011" => -- OR
26                 result <= op_a or op_b;
27             when "0100" => -- XOR
28                 result <= op_a xor op_b;
29             when "0101" => -- NOT A
30                 result <= not op_a;
31             when others =>
32                 result <= (others => '0');
33         end case;
34     end process;
35 end Behavioral;

```

Listing 1: Descripción VHDL de la ALU de 4 bits (alu1_0.vhd).

Nota: La implementación utiliza un control de 4 bits. En esta práctica se validaron en hardware las operaciones mostradas en la Tabla 3.

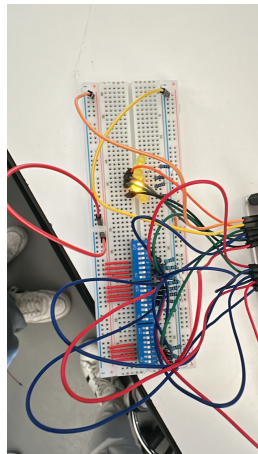
6.2. Pruebas experimentales en FPGA

Se validaron operaciones en hardware. En todos los casos, la salida observada en los LEDs coincidió con el resultado esperado.

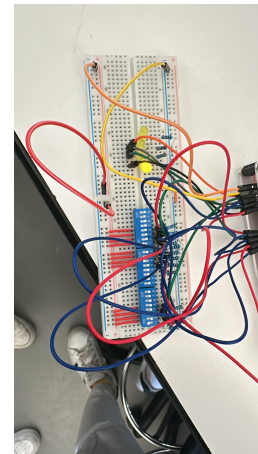
Cuadro 3: Resumen de pruebas realizadas en la PYNQ-Z1.

Operación	alu_ctrl	Entradas (A , B)	Resultado esperado	Validación
ADD	0000	1, 1	2 (0010)	Correcto
SUB	0001	2, 1	1 (0001)	Correcto
AND	0010	1, 1	1 (0001)	Correcto
OR	0011	3, 1	3 (0011)	Correcto
XOR	0100	3, 1	2 (0010)	Correcto
NOT	0101	3, –	12 (1100)	Correcto

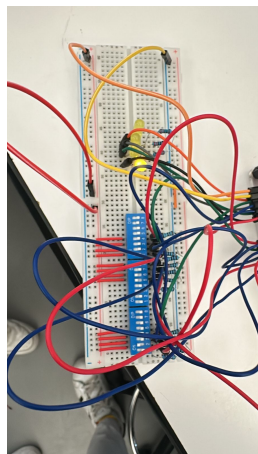
6.3. Evidencia fotográfica



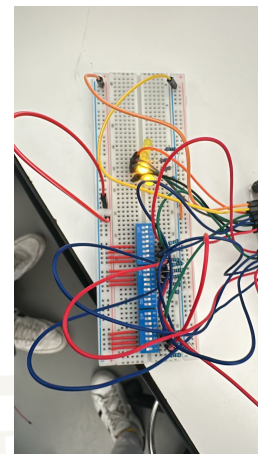
(a) ADD



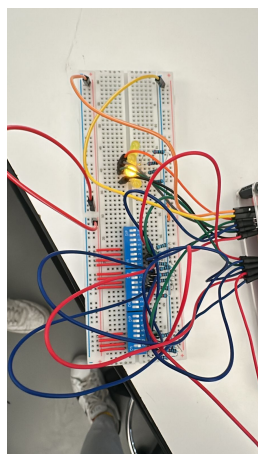
(b) SUB



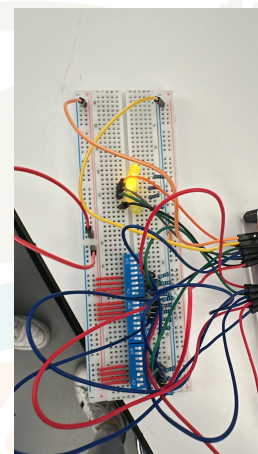
(c) AND



(d) OR



(e) XOR



(f) NOT

Figura 1: Evidencias de las pruebas de la ALU en la PYNQ-Z1 con protoboard.

6.4. Discusión

Los puntos críticos durante la implementación fueron:

1. **Definición del top module:** Vivado requiere que el módulo principal esté correctamente identificado para generar el bitstream.
2. **Consistencia VHDL–XDC:** Los nombres de puertos en el código VHDL deben coincidir exactamente con los del archivo de restricciones.
3. **Maapeo físico PMOD:** Se debe respetar la numeración de pines y la lógica.

7. Comparación teórico-práctica

Los resultados obtenidos en hardware coinciden con los valores teóricos calculados para cada operación. Por ejemplo, para la suma $1 + 1 = 2$, los LEDs muestran 0010, que corresponde al valor decimal 2. Para la operación NOT sobre $A = 3$ (0011), se obtiene 1100 = 12, que es el complemento a uno esperado en 4 bits.

8. Conclusiones

Se diseñó e implementó exitosamente una ALU de 4 bits sobre la FPGA PYNQ-Z1, validando operaciones aritméticas (ADD, SUB) y lógicas (AND, OR, XOR, NOT) en hardware real.

9. Referencias

Referencias

- [1] Práctica: Unidad Aritmética Lógica (ALU) Simple de 4 bits. Arquitecturas Computacionales, Universidad de las Américas Puebla, 2026.
- [2] RISC-V International. Especificación de la arquitectura RISC-V. Disponible en: <https://riscv.org/technical/specifications/>
- [3] IEEE. IEEE Standard VHDL Language Reference Manual (IEEE Std 1076-2008).
- [4] Digilent. PYNQ-Z1 Reference Manual. Disponible en: <https://digilent.com/reference/programmable-logic/pynq-z1/start>

- [5] AMD Xilinx. Vivado Design Suite User Guide. Disponible en: <https://www.xilinx.com/support/documentation-navigation/design-hubs/dh0002-vivado-design-hub.html>

